Control Divergence Analysis (Single and Nested Kernels)

How Nsight Compute identifies control divergence

Nsight Compute captures divergence most directly through branch and warp-efficiency counters:

- **Source Counters â†’ Avg. Divergent Branches**
- **Source Counters â†’ Branch Efficiency**
- **Warp State Statistics â†’ Avg. Active Threads Per Warp**
- **Warp State Statistics â†’ Avg. Not Predicated Off Threads Per Warp**

When divergence is present, branch efficiency drops, divergent branch count rises above zero, and the effective active threads per warp drops below 32.

Single-branch use case: Diverge vs No-Diverge

Diverge configuration (`single_branch_kernel`)

- **Avg. Divergent Branches:** 195.31
- **Branch Efficiency:** 85.71%
- **Avg. Active Threads Per Warp:** 22.75
- **Avg. Not Predicated Off Threads Per Warp:** 22.31
- **Theoretical Occupancy:** 100%
- **Achieved Occupancy:** 94.84%
- **Theoretical Active Warps Per SM:** 32
- **Achieved Active Warps Per SM:** 30.35

No-diverge configuration (`single_nodiverge_kernel`)

- **Avg. Divergent Branches:** 0
- **Branch Efficiency:** 100%
- **Avg. Active Threads Per Warp:** 32
- **Avg. Not Predicated Off Threads Per Warp:** 31.52
- **Theoretical Occupancy:** 100%

- **Achieved Occupancy:** 92.75%
- **Theoretical Active Warps Per SM:** 32
- **Achieved Active Warps Per SM:** 29.68

Interpretation

The single-branch divergent run shows the expected divergence signature:

- Non-zero divergent branches
- Lower branch efficiency
- Reduced active threads per warp

The no-diverge run is near ideal warp coherence (32 active threads, 0 divergent branches).

Nested-branch use case: Diverge vs No-Diverge

Diverge configuration (`nested_branch_kernel`)

- **Avg. Divergent Branches:** 10,525.24
- **Branch Efficiency:** 85.76%
- **Avg. Active Threads Per Warp:** 8.95
- **Avg. Not Predicated Off Threads Per Warp:** 8.78
- **Theoretical Occupancy:** 100%
- **Achieved Occupancy:** 81.48%
- **Theoretical Active Warps Per SM:** 32
- **Achieved Active Warps Per SM:** 26.07

No-diverge configuration (`nested_nodiverge_kernel`)

- **Avg. Divergent Branches:** 0
- **Branch Efficiency:** 100%
- **Avg. Active Threads Per Warp:** 32
- **Avg. Not Predicated Off Threads Per Warp:** 31.84
- **Theoretical Occupancy:** 100%
- **Achieved Occupancy:** 95.58%
- **Theoretical Active Warps Per SM:** 32
- **Achieved Active Warps Per SM:** 30.59

Interpretation

The nested divergent run exhibits very strong divergence behavior:

- Very high divergent branch activity
- Significant reduction in active threads per warp
- Lower achieved occupancy and active warps per SM compared to no-diverge

---

Other GPU Speed of Light metrics that changed

Yes, several general GPU metrics changed between divergent and non-divergent runs.

Single-branch

- **Duration:** 76.90 us (diverge) vs 51.71 us (no-diverge)
- **Memory Throughput (%):** 29.84% (diverge) vs 43.14% (no-diverge)
- **Compute (SM) Throughput (%):** 26.18% (diverge) vs 24.87% (no-diverge)

Nested-branch

- **Duration:** 1.82 ms (diverge) vs 134.30 us (no-diverge)
- **Memory Throughput (%):** 4.19% (diverge) vs 56.16% (no-diverge)
- **Compute (SM) Throughput (%):** 30.47% (diverge) vs 30.19% (no-diverge)

Overall takeaway

- **Theoretical occupancy** can remain high in both cases because it is resource-limit based.
- **Achieved occupancy, active warps, branch efficiency, and active threads per warp** are the practical indicators of control-flow efficiency.
- Divergence is clearly identifiable in your runs through the branch and warp-state counters.